

Вводная теория для менторинга: сжатие и ускорение малых языковых моделей

Oct 7, 2026 · @Diana

Как пользоваться документом

Документ даёт научному руководителю минимум теории, чтобы понимать, что делает студент, и проверять, можно ли верить его результатам. Он не заменяет статьи: ссылки на первоисточники собраны в последней секции.

Структура: сначала механика инференса и три метода (квантование, LoRA/QLoRA, прунинг), затем инструменты и метрики, затем протокол валидации, список красных флагов и план встреч. Числа из литературы приведены как ориентиры (по пересказам статей), перед цитированием в отчёте их нужно сверять с оригиналами.

Главная идея проекта: LoRA и QLoRA отвечают на вопрос «как адаптировать модель под задачу», а квантование и прунинг отвечают на вопрос «как сделать модель меньше и быстрее». Итоговый результат проекта — фронт Парето «качество, скорость, память» на конкретном железе, а не один лучший метод.

Почему SLM и зачем сжатие

Малая языковая модель (SLM) — это модель на один-несколько миллиардов параметров (в проекте 1-3B), которую можно запустить на одной потребительской видеокарте или ноутбуке. Польза от таких моделей в трёх вещах: дешёвый и быстрый инференс, запуск локально без облака (конфиденциальность, офлайн), доступное дообучение под узкую задачу.

У таких моделей есть цена: её заметно меньше ёмкость, поэтому на сложных задачах (математика уровня MATH, нетривиальный код) они сильно отстают от больших, а сжатие ещё ухудшает качество. Смысл проекта — измерить, сколько качества мы платим за каждый выигранный гигабайт памяти и каждый выигранный токен в секунду.

Потребительское железо задаёт три ограничения: объём видеопамяти (VRAM, обычно 6-12 ГБ), пропускную способность памяти (десятки-сотни ГБ/с) и доступные форматы вычислений на конкретной карте. Третье важно: не каждый формат сжатия реально ускоряется на любой карте.

Основы инференса LLM

Генерация делится на две фазы с разным узким местом, и это ключ к пониманию всего проекта.

- **Prefill** — обработка всего промпта за один проход. Операций много, фаза упирается в вычисления (compute-bound). Её время — это задержка до первого токена (TTFT).
- **Decode** — потоковая генерация по одному токenu. На каждый токен нужно заново прочитать из памяти почти все веса, поэтому фаза упирается в пропускную способность памяти (memory-bandwidth-bound). Большая часть времени ответа при коде и математике уходит именно туда, так как генерации длинные.

Потому сжатие весов обычно ускоряет decode: меньше байт читать на токен. Но ускорение реально, только если ядро деквантования на этой карте достаточно эффективно.

Память: что занимает VRAM

Память под веса — число параметров, умноженное на байты на параметр:

$$M_{\text{weights}} \approx N_{\text{params}} \cdot \frac{b}{8}$$

где b — биты на вес (FP16 = 16, 8-бит = 8, 4-бит $\approx$ 4,5–5 с учётом масштабных коэффициентов). Для модели 3B: FP16 $\approx$ 6 ГБ, 8-бит $\approx$ 3 ГБ, 4-бит $\approx$ 1,7–2 ГБ.

Кроме весов нужны KV-кэш (ключи и значения внимания для уже обработанных токенов), активации и служебные буферы:

$$M_{\text{KV}} \approx 2 \cdot L \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot T \cdot B \cdot \frac{b_{\text{kv}}}{8}$$

где L — число слоёв, n_{kv} — число KV-голов (у современных моделей с GQA оно меньше числа голов запросов), d_{head} — размер головы, T — длина контекста, B — размер батча. На длинных контекстах и больших батчах KV-кэш может быть сопоставим с весами, а веса сжатием весов не лечатся.

Оценка потолка скорости decode

Грубый верхний предел скорости при батче 1:

$$v_{\text{decode}} \lesssim \frac{BW}{M_{\text{weights}}}$$

где BW — пропускная способность памяти. Пример (порядка величин, не замер): для карты с пропускной способностью около 300 ГБ/с модель 3B в FP16 (6 ГБ) даёт потолок порядка 50 токенов/с, а в 4-битном виде (≈ 2 ГБ) — порядка 150 токенов/с. Реальные замеры всегда ниже из-за накладных расходов, деквантования и других слоёв. Если студент показывает скорость больше этого потолка — это красный флаг (промпт закэширован, батч больше 1 или ошибка в замерах).

Квантование

Квантование заменяет веса в FP16 на целые числа меньшей разрядности (8, 6, 5, 4, 3 бита) и набор масштабных коэффициентов. Основная формула — равномерное квантование с масштабом s и нулевой точкой z :

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{w}{s}\right) + z, \, 0, \, 2^{b-1}\right), \quad \hat{w} = s \cdot (q - z)$$

Веса квантуются группами (обычно 32–128 весов на один масштаб): чем мельче группа, тем точнее, но тем больше накладных расходов. Поэтому «4-битная» модель на деле занимает около 4,5–5 бит на вес.

Главное различие методов — как подбираются коэффициенты. Простое округление (RTN) быстро, но теряет качество на низких битах; GPTQ корректирует оставшиеся веса по калибровочным данным; AWQ защищает веса, важные по активациям; NF4 (из QLoRA) использует неравномерную сетку, рассчитанную на нормальное распределение весов.

Формат	Где работает	Типичное применение	На что смотреть
GGUF (k-quants: Q8_0, Q6_K, Q5_K_M, Q4_K_M, Q3_K_M)	llama.cpp: CPU, GPU, Apple Silicon	Локальный запуск на любом железе	Размер файла, скорость на CPU и GPU, провал на Q3 и ниже
AWQ (4-бит)	CUDA; vLLM, Transformers	Серверный и настольный GPU-инференс	Калибровочный набор, версия кернала, поддержка архитектуры модели

Формат	Где работает	Типичное применение	На что смотреть
GPTQ (4/8-бит)	CUDA; Transformers, vLLM, ExLlama	GPU-инференс	На старых GPU может быть медленнее FP16 из-за деквантования
NF4 / FP4 (bitsandbytes)	CUDA	Обучение в QLoRA, не лучший вариант для скорости	Память экономится, скорость не обязательно
FP8 / INT8 (W8A8)	Новые GPU (FP8 — с архитектуры Ada/Hopper)	Сервисы на современных картах	Нужна поддержка аппаратуры

Что ожидать по качеству (из литературы, ориентиры): 8-бит почти без потерь; 5–6 бит очень близко к FP16; 4 бита — заметная, но часто приемлемая просадка, особенно на малых моделях и сложных задачах; 3 бита и ниже — часто критично. Математика и длинные цепочки рассуждений чувствительнее к квантованию, чем код и выбор ответа из вариантов: в работе Quantization Meets Reasoning на GSM8K просадка в среднем около 7%, а на MATH у модели 0,5B с GPTQ доходила до $\approx 70\%$. На моделях Qwen3 замечена более сильная деградация, чем у прошлых поколений, при 3 битах и ниже.

Про скорость: квантование экономит память всегда, но ускоряет не всегда. В одной из работ GPTQ на GPU T4 сократил память на 41%, но замедлил генерацию, а GGUF Q4_K_M на CPU дал многократное ускорение. Поэтому выводы о скорости допустимы только для той связки «железо + фреймворк + формат», на которой они измерены.

LoRA и QLoRA

LoRA дообучает не все веса, а небольшую низкоранговую поправку к замороженным весам. Для матрицы W размера $d \times k$ обучаются две матрицы B и A ранга r (r много меньше d и k):

$$W' = W + \frac{\alpha}{r} B A, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

Обучаемых параметров обычно около 0,1–2% от модели, поэтому память под градиенты и состояния оптимизатора резко падает. QLoRA добавляет ещё и 4-битное хранение базовой модели (NF4 с двойным квантованием масштабов и

paged-оптимизаторами), чтобы дообучать модели 1-3B на карте с небольшим объёмом памяти.

Важно не путать роль: LoRA — это адаптация под задачу, а не сжатие. После обучения адаптер можно слить с базовой моделью и уже итоговую модель квантовать. Важное тонкое место: слияние адаптера, обученного на 4-битной базе, с базой в FP16 даёт чуть другую модель, чем видела во время обучения, поэтому качество нужно мерить уже на финальном слитом и квантованном артефакте. Похожие идеи для того, чтобы согласовать адаптацию и квантование, — QA-LoRA, LoftQ и L4Q.

Гиперпараметры, которые студент должен уметь объяснить и зафиксировать:

- **Ранг r и коэффициент α** — ёмкость адаптера и масштаб поправки (типично $r = 8-64$). Большой r не гарантирует роста качества.
- **Какие слои адаптируются** — только внимание или все линейные слои (внимание и MLP). Обычно все линейные дают лучшее качество.
- **Learning rate, число эпох, длина последовательности, размер батча и накопление градиентов** — влияют и на качество, и на потребление памяти.
- **Данные** — источник, фильтрация, дедупликация, шаблон промпта. Чаще всего именно они решают результат.
- **Правило честности:** тренировочные данные не должны пересекаться с тестовыми бенчмарками (HumanEval, MBPP, GSM8K, MATH). Нужна проверка на перекрытие.

Прунинг

Прунинг удаляет часть весов или структурных единиц модели. Его главная особенность для проекта: «меньше нулей по параметрам» не равно «быстрее в практике».

- **Неструктурный** (Wanda, SparseGPT): обнуляет отдельные веса по критерию важности (у Wanda — модуль веса, умноженный на норму активации). Матрицы остаются того же размера, поэтому память и скорость без специальных разреженных ядер не меняются. Часто говорят о шаблоне 2:4 (из каждых четырёх весов остаются два): он может дать ускорение на картах с аппаратной поддержкой (начиная с Ampere), но требует поддержки в фреймворке и часто теряет качество.
- **Структурный** (удаление каналов MLP, голов внимания, целых слоёв): реально уменьшает матрицы. Варианты: **depth** (удалить слои, лучше подряд идущие) и **width** (сузить скрытые размерности). В Minitron удаление слоёв

дало ускорение до $\approx 2,7\times$, сужение по ширине — до $\approx 1,8\times$ (H100), но качество у width-варианта обычно лучше.

Работа Minitron требует после прунинга дистилляции на сотнях миллиардов токенов, что на слабом железе невыполнимо. В проекте реалистичны только пост-тренировочный неструктурный прунинг и лёгкий структурный (несколько слоёв) с коротким восстановлением через LoRA.

Что показывает литература:

- На SLM 0,5–3,8B квантование стабильнее прунинга при сопоставимом сжатии: на уровне 50% квантование сохраняло $\approx 98\%$ точности, прунинг — $\approx 83\%$, причём неструктурный прунинг сильнее всего бьёт по не-английским языкам.
- Даже небольшой структурный прунинг может не ускорить, а замедлить инференс на CPU: удалённая ширина ломает выравнивание блоков квантования, и файл даже раздувается. В этой же работе LoRA восстановила качество только частично.
- Даже правильный прунинг даёт ускорение только если собранная модель действительно меньше и фреймворк это использует. Всегда нужны замеры размера файла и tokens/s.

Инструменты и метрики

Практический стек проекта достаточно стабилен: Hugging Face Transformers + PEFT + TRL (или Unsloth) для дообучения; bitsandbytes, AutoAWQ/GPTQModel и llama.cpp для квантования; vLLM и llama.cpp для инференса и быстрых массовых прогонов; EvalPlus и lm-evaluation-harness для оценки; WandB для логов. Версии всех библиотек должны быть зафиксированы.

Метрика	Что измеряет	Как мерить	Подводные камни
pass@1 (HumanEval+, MBPP+)	Функциональная корректность кода	EvalPlus; жадная генерация или фиксированный seed и температура	Малая выборка (164 задачи у HumanEval), разные шаблоны промпта, утечки

Метрика	Что измеряет	Как мерить	Подводные камни
Точность на GSM8K / MATH-500	Математическое рассуждение	Im-eval или собственный скрипт с явным парсингом ответа	Ошибки парсинга ответа, длина цепочки, обрезанные ответы
Перплексия (WikiText2, C4)	Общее качество языкового моделирования	быстрый тест на размещение	Хорошо коррелирует не всегда с реальной задачей; дополняет, не заменяет
Decode tokens/s	Скорость потоковой генерации	Фиксированная длина входа и выхода, несколько повторов, медиана	Нагрев карты, прогрев кэшей, фоновые процессы
Prefill tokens/s и TTFT	Обработка промпта, задержка до первого токена	Длинный и короткий промпт отдельно	Смешивание с decode
Пиковая VRAM	Реальный расход памяти включая KV-кэш	nvidia-smi, torch.cuda.max_memory_allocated	Разные длины контекста и размеры батча
Размер артефакта	Файл на диске	Байты файла	Прунинг может раздуть файл

Метрика	Что измеряет	Как мерить	Подводные камни
Длина вывода	Модель может стать многословнее после сжатия	Среднее число токенов на ответ	Если токенов больше, время на задачу растёт, даже если tokens/s выше

Рекомендуемая итоговая метрика скорости — **время на решённую задачу**, а не только tokens/s: сжатая модель может генерировать быстрее, но дольше рассуждать. В литературе такой эффект описан как «рост числа токенов при низкобитном рассуждении», и это одна из самых интересных тем для анализа в работе.

Для сравнения конфигураций строится фронт Парето: конфигурация доминируется, если есть другая не хуже по качеству и не дороже по скорости и памяти. В трёхмерном случае удобно смотреть две проекции: «качество vs VRAM» и «качество vs время на задачу».

Протокол эксперимента и валидация результатов

Результату можно верить, если он воспроизводим, сравним по единому протоколу и подкреплён статистикой. Ниже — минимум, который стоит потребовать от студента с первой недели.

1. Базовая точка и контрольные сравнения

- Все конфигурации сравниваются с необученной моделью в FP16/BF16 на том же наборе задач и с той же шаблоном промпта.
- Для каждой адаптации нужны две базы: модель без дообучения и модель с дообучением без сжатия, иначе нельзя разделить эффект адаптации и эффект сжатия.
- Одно изменение за раз: нельзя одновременно менять модель, битность и фреймворк.

2. Воспроизводимость

- Репозиторий в Git, фиксация версий (requirements или Docker), хэш коммита к каждому эксперименту.

- Конфигурация каждого запуска в отдельном файле (модель, версия весов, квант, параметры генерации, seed), сырые ответы модели сохраняются.
- Логи в WandB или аналоге; журнал экспериментов с гипотезой, датой и выводом.
- Детерминированная генерация (жадный декодинг, температура 0) для сравнения конфигураций; для стохастической — несколько seed и среднее с разбросом. Даже жадный декодинг на GPU может давать небольшие различия между запусками, это нужно измерить и указать.

3. Контроль утечек и честности

- Разделение train / validation / test; гиперпараметры подбираются только на validation, тест смотрится один раз в конце.
- Проверка пересечения обучающих данных с бенчмарками (точные совпадения и n-gram). Публичные бенчмарки могут быть в предобучении самой модели, поэтому без измерения на дополнительном «свежем» наборе (например, свои задачи или новые подмножества) выводы остаются осторожными.
- Калибровочные данные для AWQ, GPTQ и Wanda не должны включать тестовые задачи; указывается их источник и размер.

4. Статистика

Разница в два-три процентных пункта на бенчмарке из сотен задач часто не значима. Для доли верных ответов p на n задачах стандартная ошибка:

$$SE = \sqrt{\frac{p(1-p)}{n}}$$

Пример: $p = 0,40$ на HumanEval ($n = 164$) даёт $SE \approx 0,038$, то есть 95%-й интервал порядка $\pm 7,5$ п.п. Разница в 3–5 п.п. между двумя квантами на одних и тех же 164 задачах — в пределах шума, если не доказано обратное.

- Оценки даются с доверительными интервалами (бутстрап по задачам или интервал Уилсона).
- Конфигурации сравниваются на одинаковых задачах парным тестом (МакНемара или парный бутстрап), а не по голым средним.
- Если стохастический декодинг — не менее 3–5 прогонов и среднее $\pm$ разброс. При одном прогоне обязательны осторожные формулировки.
- Для скорости — минимум 5–10 повторов после прогрева, медиана и разброс, фиксированные длины входа и выхода, одинаковые драйвер и тактовые частоты, отключённые фоновые процессы, контроль температуры.

- Множественные сравнения: чем больше конфигураций, тем выше шанс «найти» случайный выигрыш. Такие выводы нужно подтверждать на втором наборе задач.

5. Валидация качества помимо одного числа

- Ручной разбор 20–50 ответов каждого типа (верные, неверные, обрезанные): проверяется, не считает ли скрипт правильные ответы неверными и наоборот.
- Для кода: кроме `pass@1`, таймауты, ошибки синтаксиса, доля пустых или зацикленных ответов.
- Для математики: доля ответов, не дошедших до финального числа, и типы ошибок (вычисления, интерпретация, начало цепочки).
- Для бинарных и классификационных задач: сбалансированная точность и распределение предсказаний, чтобы не дать модели спрятаться за одним ответом.